AI-Powered Vibe Coding with GitHub Copilot

AI Workflows for Biomedical Data Science

Anthony Christidis
Computational Scientist

Alex Pickering
Sr. Software Engineer

Core for Computational Biomedicine (CCB)
Department of Biomedical Informatics (DBMI)
Harvard Medical School (HMS)

April 14, 2026

HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Workshop Materials

Everything we do today is open-source.

- Slides
- Setup Guides
- Live Demo Scripts
- Hands-On Datasets & Starter Code

GitHub Repository:

https://github.com/ccb-hms/ai-powered-coding

Workshop Agenda (120 Minutes)

Part 1: Presentation & Core Concepts **45 min**

The AI Landscape • Context Memory • Prompting Strategies • Data Privacy

---

Part 2: Live Demonstration **45 min**

Copilot in Action • Code Refactoring • Error Debugging • IDE Tools

---

Part 3: Hands-On Exercise **30 min**

TCGA Survival Analysis • Guided AI Challenges • Instructor Support

What we *mean* by "Vibe Coding"

"Focusing on the **overall outcome** *(the vibe) rather than writing syntax line-by-line."*

— Coined by Andrej Karpathy (2025)

The Approach:

- **Natural Language:** Prompting AI to generate, debug, and iterate.
- **Rapid Prototyping:** Prioritizing speed and creativity to test ideas quickly.
- **Role Shift:** The developer becomes a **director**, not a typist.

What we *don't* mean by "Vibe Coding"

The Golden Rule of AI Coding

You cannot just "go with the vibe."

You remain **strictly accountable** for understanding and validating all AI-generated code.

Important Considerations:

No Blind Acceptance • Standards Still Apply • Prototyping $\neq$ Production

The Vibe Coding Mindset: Offloading vs. Surrender

The Goal: Focus on scientific intent, not syntax.

Cognitive Offloading (GOOD)

- **AI is the typist.**
- Offload boilerplate, syntax, and formatting.
- Frees mental space for high-level scientific logic.

Cognitive Surrender (BAD)

- **AI is the scientist.**
- Blindly trusting AI-generated math or logic.
- Running code you cannot read or explain.

You remain the pilot. Copilot is just the navigator.

The AI Coding Landscape

Copilot is the enterprise standard, but it is not alone.

Cursor

- Standalone AI editor.
- Top choice for power users.

GitHub Copilot

- Enterprise standard.
- Now supports local/offline models for privacy.

Continue (.dev)

- Open-source extension.
- Connects to any custom or local AI model.

Choosing Your Environment

VS Code

- The industry standard.
- Offers the most stable, full-featured Copilot experience.

Positron

- Uses *Positron Assistant*.
- **Superpower:** Reads your live R/Python environment (dataframes!), not just text.

RStudio

- **Stable Release:** Copilot autocomplete only.
- **Daily Builds:** The full Posit AI Assistant was just added in early beta!

What is GitHub Copilot Today?

It is no longer just "autocomplete." It is an umbrella of integrated tools:

1. **Code Completion** (Ghost Text)
2. **Copilot Chat** (Sidebar & Inline)
3. **Copilot Agents** (Autonomous edits)

Plus: You can now choose your underlying AI model (Claude Sonnet 4.6, GPT-5.4, etc.).

The Copilot Interface

Explain what the #sym:calculate_log2_fc function does.

calculate_log2_fc utils.R

The function `calculate_log2_fc` in utils.R:7-14 computes the log2 fold change between two numeric vectors, typically representing treatment and control groups in an experiment.

Here's how it works:

1. It calculates the mean of the `treatment_vals` vector, ignoring any NA values.
2. It calculates the mean of the `control_vals` vector, also ignoring NA values.
3. If the control mean is NA or zero (to avoid division by zero or undefined results), it returns NA.
4. Otherwise, it computes the log base 2 of the ratio of the treatment mean to the control mean: $\log_2\left(\frac{\text{treatment mean}}{\text{control mean}}\right)$.

This value represents the log2 fold change, a common metric in biology for quantifying the relative change between two conditions.

utils.R

Ask a question about your code or project

Ask GPT-4.1

Local Default Approvals

The Old Way: Browser Chat (ChatGPT / Claude Web)

- **Isolated:** The AI has no idea what is on your computer.
- **Tedious:** Requires constant copy/pasting of your code, column names, and errors.
- **Inaccurate:** Frequently hallucinates variable names because it lacks context.

The Old Way: Manual Context

Good afternoon, Anthony Christidis

Hello! I'm GPT-5.1, running in the Harvard AI Sandbox. I have a 400k token context window (~300k words) and my knowledge cutoff is September 2024. I also have the ability to search the web in real time to access the most up-to-date information, something that most other models may not support. How can I help you today?

I need to calculate the mean biomarker score for each treatment group and make a boxplot.

Here are the exact columns in my biomarker_dummy.csv file:
patient_id, age, treatment_group, biomarker_score, response_status.

Here is the code inside my utils.R script that I need to use:

```
calculate_log2_fc <- function(values, control_values) {
  mean_val <- mean(values, na.rm = TRUE)
  mean_ctrl <- mean(control_values, na.rm = TRUE)
  if (mean_ctrl == 0) return(NA)
  return(log2(mean_val / mean_ctrl))
}
```

And here is my current working script (live_demo.R):

```
library(dplyr)
library(ggplot2)
source("utils.R")
df <- read.csv("biomarker_dummy.csv")
```

How do I combine all of this to get my summary table and plot?

File Search

The New Way: IDE Integration (GitHub Copilot)

- **Context-Aware:** The AI lives *inside* your workspace.
- **Omniscient:** It automatically sees your open tabs, folders, and highlighted text.
- **Accurate:** Writes code that perfectly matches *your* environment and variable names.

The New Way: Automatic Context

How do I use the functions in to summarize biomarker_score by treatment_group from biomarker_dummy.csv and add a boxplot to live_demo.R. Read the files then tell me.

Reviewed 3 files

Read biomarker_dummy.csv, lines 1 to 20

Read live_demo.R, lines 1 to 40

Read utils.R

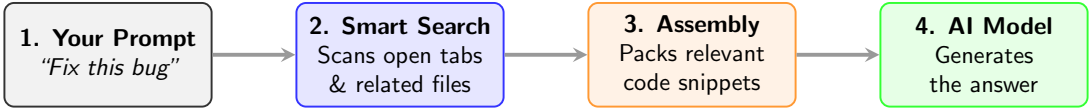
How Copilot Works: The Context Window

- **Myth:** *"Copilot reads my entire project every time I type."*
- **Reality:** While modern AI has huge memory, feeding it thousands of irrelevant files causes hallucinations and slows it down.

The Solution: Copilot selectively curates a limited **"Context Window"** to stay fast and accurate.

How Copilot Works: Smart Search

Copilot acts as an incredibly intelligent search engine:



The AI is remarkably good at finding the right files automatically.
But when it occasionally misses one, we take manual control!

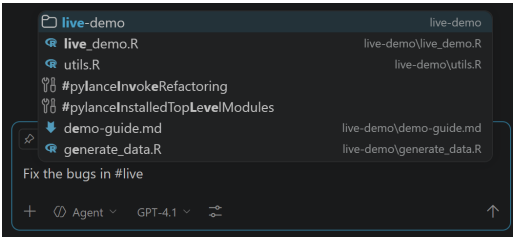
Best Practice: Pinning Context with

Copilot's automatic search is incredibly smart...

Modern models automatically scan your entire workspace, terminal, and related files.

...but sometimes it needs a hint.
Use the # symbol to manually pin specific context:

- `#file` (e.g., `#utils.R`)
- `#selection` (Highlighted code)
- `#symbol` (Specific functions)



Typing # opens the context menu

Interacting with AI: Three Sidebar Modes

- **1. Chat:** The AI acts as a tutor. It explains concepts and generates code snippets for you to manually copy and paste.
- **2. Plan:** The AI acts as an architect. It writes a step-by-step logic blueprint *before* generating any code.
- **3. Agent:** The AI acts as a junior developer. It autonomously edits multiple scripts, writes code, and runs terminal commands.

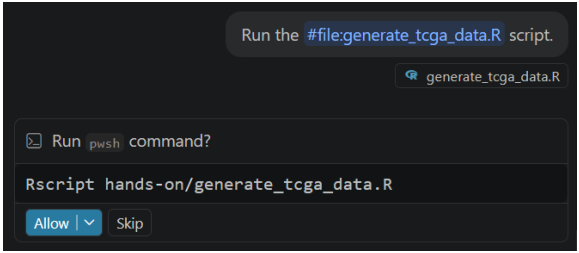
Copilot manages your environment, not just your script.

1. Seamless Debugging

- Automatically reads console errors.
- No copy/pasting required.

2. Agentic Execution

- Can autonomously use the terminal.
- Installs packages and runs scripts for you.



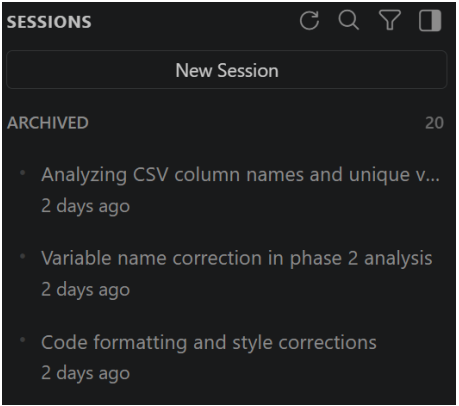
Agent Mode asking permission to run a terminal command

Managing Context: Multiple Sessions

Keep your conversations focused by starting fresh.

Isolated Context Windows

- Click the + icon in the chat panel to open a new session.
- Every session has its own **independent memory**.
- Prevents the AI from confusing your R project with your Python project!



Customizing Copilot: The Power of "Rules Files"

Stop typing "don't yap" in every single prompt.

- **The Problem:** AI forgets your style during long sessions (Context Rot).
- **The Fix:** Type `/init` to auto-generate a `copilot-instructions.md` file.
- **The Rulebook:** Silently applies your global preferences to *every* prompt.

Global Rules in Action

ai-powered-coding / .github / copilot-instructions.md

AnthonyChristidis Simplify copilot instructions by removing coding standards and reprod... ee8bbbd · now History

Preview Code Blame Raw

Global AI Instructions for CCB Workshop

1. **Tone & Formatting:**
   - Output only the necessary code or direct answers.
   - Do not apologize or add conversational filler (e.g., "Certainly! Here is the code...").
   - Ensure all generated code is highly readable and well-commented.
2. **Data Privacy & Security:**
   - You are a security-aware assistant working in a biomedical research environment.
   - Never hardcode sensitive information (e.g., passwords, API keys, or PHI) into scripts.
   - Always assume input data paths are local relative paths (e.g., `data/` or `results/`), not absolute machine paths.

Targeted Rules: `/create-instructions`

Apply specific rules only when you need them.

- **The Problem:** Global rules fail in mixed projects (e.g., forcing `dplyr` in Python files).
- **The Command:** Type `/create-instructions` to build context-specific rulebooks.
- **Smart Routing:** Rules only activate for matching files using `applyTo: "*.R"`.

Targeted Rules in Action

master ai-powered-coding / .github / instructions / nice-comments.instructions.md

Go to file t

AnthonyChristidis Remove redundant persona and trim instruction/agent metadata for brevity 6806e17 · now History

Preview Code Blame Raw

description	Always use clear, informative, and friendly comments in all R code.
applyTo	**/*.R **/*.rmd **/*.qmd
rule	- Every function, data wrangling step, and plot must be preceded by a concise, meaningful comment.

Instruction: Use Nice Comments

All R code in this project must include clear, helpful comments that explain the purpose and logic of each step. Comments should be friendly, professional, and guide the reader through the analysis. Avoid stating the obvious or repeating code in comments.

Custom Personas: `/create-agent`

Build a specialized AI teammate for your lab in seconds.

- **The Wizard:** Type `/create-agent` in chat to start the interactive builder.
- **Plain English:** Describe what you need *(e.g., "I need an R survival expert").*
- **Reusable File:** Auto-generates an `.agent.md` file. Summon anytime with `@Name`.

The Interactive Agent Builder

ai-powered-coding / .github / agents / **r-survival-expert.agent.md**

AnthonyChristidis Remove redundant persona and trim instruction/agen... 6806e17 · 1 minute ago History

Preview Code Blame Raw

name	R Survival Analysis Expert			
tool_preferences	survminer for plots	dplyr for data	Always check NAs	Never us
scope	- Survival analysis (Kaplan-Meier, Cox) - Clinical/biomedical data			

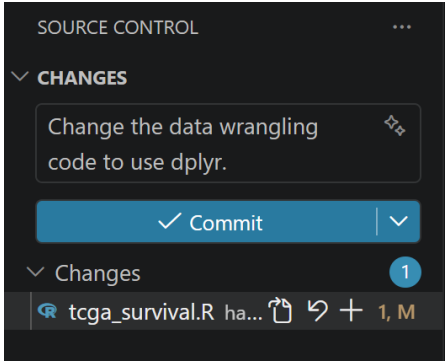
R Survival Analysis Expert Agent

R survival analysis expert. Use survminer for plots, dplyr for data, always check NAs. Never use base R for plotting/wrangling.

Stop writing “fixed bug” in your commit history.

Automated Commits

- Clicking the **sparkle** icon allows Copilot to read your exact file diffs.
- It auto-generates a highly professional, accurate commit message summarizing your work.

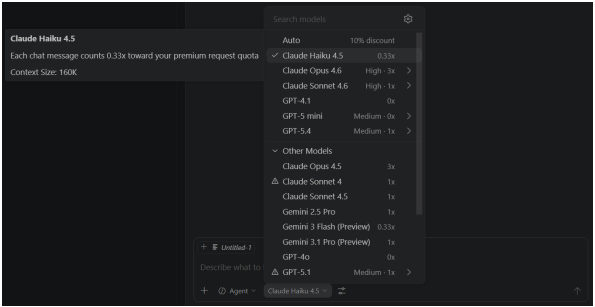


Preparing for the Hands-On: Model Selection

You are not locked into one AI. You can swap engines on the fly.

Action Required:

- Open Copilot Chat.
- Click the **Model Dropdown** in the top corner.
- Select your preferred model.

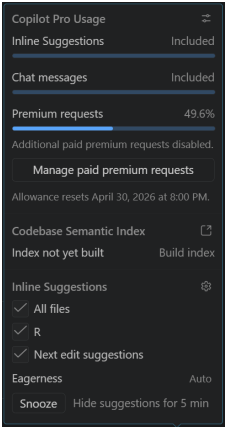


Understanding Your Monthly Quota

Copilot Pro provides a budget of 300 Premium Requests per month.

The Multiplier System:

- **Base Models:** Unlimited.
- **1x Models:** 1 request per prompt
  (Max: 300 per month)
- **0.33x Models:** 1/3 request per prompt
  (Max: 900 per month!)
- **3x Models:** 3 requests per prompt
  (Max: 100 per month)



Our Experience: Picking the Right Model

Claude Haiku 4.5 *0.33x Cost*

Our Take (The Daily Driver): Fast and handles 80% of everyday coding and debugging. Excellent for stretching your monthly quota. **Our recommendation for today's hands-on!**

Sonnet 4.6 & Gemini Pro 3.1 *1x Cost*

Our Take (The Sweet Spot): The best overall balance of intelligence and cost for data science.

Claude Opus 4.5 *3x Cost*

Our Take (The Heavyweight): Extremely smart, but burns through your monthly limit very quickly. Save it for massive, complex architecture problems.

Data Privacy: The Two Ways Data Leaves Your Machine

When you use GitHub Copilot, your code and data are sent to the cloud for two completely different reasons:

1. Model Training

GitHub wants to permanently save your code to teach its future AI models how to write better software.

Can we stop this? YES.

2. Live Inference

Copilot must temporarily send your code to a server to generate the immediate answer to your prompt.

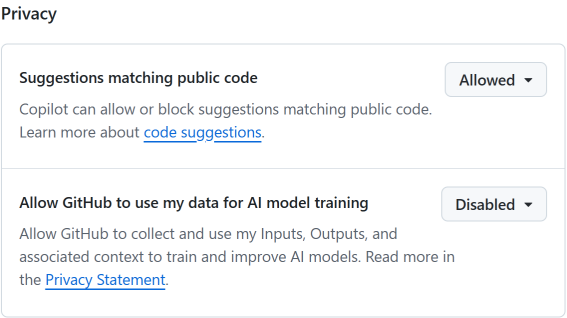
Can we stop this? NO.

Action Required: Disable Model Training

You must actively opt-out of allowing GitHub to read your data for future model training.

To secure your account:

- Go to your GitHub.com Settings.
- Navigate to the **Copilot** tab.
- **DISABLE:** *"Allow GitHub to use my data for AI model training."*



The Hard Rule: Live Inference & PHI

Even if training is disabled, your prompt is still sent to a cloud server to be processed.

Critical Security Rule

NEVER use GitHub Copilot
(or any cloud-based LLM) with:

Protected Health Information (PHI) • Personally Identifiable Info (PII) • Restricted Datasets

HARVARD MEDICAL SCHOOL | INFORMATION TECHNOLOGY

Official HMS Generative AI Policy:

it.hms.harvard.edu/about/policies-and-guidelines/generative-ai

Beyond Copilot: AI *Inside* Your Code

AI can also act as a computational engine within your scripts.

LLMs as R/Python Functions:

- **API Endpoints:** Call models (Claude, GPT) directly from your script.
- **AI Data Manipulation:** Packages like `ellmer` (Posit) tidy data or run NLP on dataframes using plain English.

Resource:

Comprehensive List of R LLM Packages

Want to learn this?

Posit AI Tools @ Harvard: Advanced AI Integration in R and Python

April 29th — Lahey Room

Upcoming Workshop Announcement

Live Demonstration

Let's see it in action.